

PBA: 3. Data Wrangling

Fall 2026

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

Where are we? Where are we going?

- **Where we have been**

- Week 1: R and RStudio, functions and packages, Quarto
- Week 2: data visualization. A table became a graph, layer by layer, with `ggplot2`

Where are we? Where are we going?

- **Where we have been**

- Week 1: R and RStudio, functions and packages, Quarto
- Week 2: data visualization. A table became a graph, layer by layer, with `ggplot2`

- **Today: data wrangling with `dplyr`**

- Building on last week: graphs need the right table first. Choose rows and columns, create variables, summarize groups, then draw barplots from the summaries
- After the lecture you run today's code yourself, then E2 at your own pace with the TAs. Assignment 1 is released today

Where are we? Where are we going?

- **Where we have been**

- Week 1: R and RStudio, functions and packages, Quarto
- Week 2: data visualization. A table became a graph, layer by layer, with `ggplot2`

- **Today: data wrangling with `dplyr`**

- Building on last week: graphs need the right table first. Choose rows and columns, create variables, summarize groups, then draw barplots from the summaries
- After the lecture you run today's code yourself, then E2 at your own pace with the TAs. Assignment 1 is released today

- **Where we are going**

- No class next Monday (Sep 28, Teachers' Day). On Oct 5: causality. Then prediction, regression, and uncertainty

Roadmap

1. Operating on Rows
2. Operating on Columns
3. Operating on Groups
4. Creating Barplots
5. In-Class Exercise E2

Data Wrangling $\rightsquigarrow$ Q: Why Do It?



Source: created using a generative AI, DALLE-3!

Prompt I used: I want to visualize how real-world data is very messy. We need data wrangling before we can model it and extract insights. Can you help create a visualization illustrating this?

data.frames vs. tibbles

- The standard R object for datasets is the `data.frame`
 - Each column is a vector of the same length
 - Columns can be different types
- Access columns with `$`: `my_data$my_variable`
- **Example:**

```
1 > mtcars$mpg
2
3 [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4
4 [17] 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7 15.0 21.4
```


Problems with Data Frames

```
1 > mtcars
2
3      mpg  cyl  disp  hp  drat    wt    qsec vs  am  gear  carb
4 Mazda RX4      21.0   6 160.0 110  3.90  2.620 16.46 0  1    4    4
5 Mazda RX4 Wag    21.0   6 160.0 110  3.90  2.875 17.02 0  1    4    4
6 Datsun 710       22.8   4 108.0  93  3.85  2.320 18.61 1  1    4    1
7 Hornet 4 Drive   21.4   6 258.0 110  3.08  3.215 19.44 1  0    3    1
8 Hornet Sportabout 18.7   8 360.0 175  3.15  3.440 17.02 0  0    3    2
9 Valiant          18.1   6 225.0 105  2.76  3.460 20.22 1  0    3    1
10 Duster 360       14.3   8 360.0 245  3.21  3.570 15.84 0  0    3    4
11 Merc 240D        24.4   4 146.7  62  3.69  3.190 20.00 1  0    4    2
12 Merc 230         22.8   4 140.8  95  3.92  3.150 22.90 1  0    4    2
13 Merc 280         19.2   6 167.6 123  3.92  3.440 18.30 1  0    4    4
14 Merc 280C        17.8   6 167.6 123  3.92  3.440 18.90 1  0    4    4
15 Merc 450SE       16.4   8 275.8 180  3.07  4.070 17.40 0  0    3    3
16 Merc 450SL       17.3   8 275.8 180  3.07  3.730 17.60 0  0    3    3
17 Merc 450SLC      15.2   8 275.8 180  3.07  3.780 18.00 0  0    3    3
18 Cadillac Fleetwood 10.4   8 472.0 205  2.93  5.250 17.98 0  0    3    4
19 Lincoln Continental 10.4   8 460.0 215  3.00  5.424 17.82 0  0    3    4
20 Chrysler Imperial 14.7   8 440.0 230  3.23  5.345 17.42 0  0    3    4
21 Fiat 128         32.4   4  78.7  66  4.08  2.200 19.47 1  1    4    1
22 Honda Civic      30.4   4  75.7  52  4.93  1.615 18.52 1  1    4    2
23 Toyota Corolla   33.9   4  71.1  65  4.22  1.835 19.90 1  1    4    1
24 Toyota Corona    21.5   4 120.1  97  3.70  2.465 20.01 1  0    3    1
25 Dodge Challenger 15.5   8 318.0 150  2.76  3.520 16.87 0  0    3    2
26 AMC Javelin      15.2   8 304.0 150  3.15  3.435 17.30 0  0    3    2
27 Camaro Z28       13.3   8 350.0 245  3.73  3.840 15.41 0  0    3    4
28 Pontiac Firebird  19.2   8 400.0 175  3.08  3.845 17.05 0  0    3    2
29 Fiat X1-9         27.3   4  79.0  66  4.08  1.935 18.90 1  1    4    1
30 Porsche 914-2    26.0   4 120.3  91  4.43  2.140 16.70 0  1    5    2
31 Lotus Europa     30.4   4  95.1 113  3.77  1.513 16.90 1  1    5    2
```

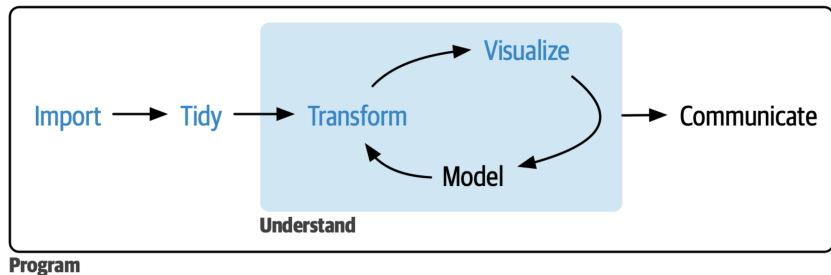
tibbles: A Tidyverse Alternative

```
1 > midwest
2 # A tibble: 437 × 28 rows x columns
3   PID county state area poptotal popdensity popwhite popblack popamerindian
4   <int> <chr> <chr> <dbl> <int> <dbl> <int> <int> <int> column
5 1 561 ADAMS IL 0.052 66090 1271. 63917 1702 98 types
6 2 562 ALEXANDER IL 0.014 10626 759 7054 3496 19
7 3 563 BOND IL 0.022 14991 681. 14477 429 35
8 4 564 BOONE IL 0.017 30806 1812. 29344 127 46
9 5 565 BROWN IL 0.018 5836 324. 5264 547 14
10 6 566 BUREAU IL 0.05 35688 714. 35157 50 65
11 7 567 CALHOUN IL 0.017 5322 313. 5298 1 8
12 8 568 CARROLL IL 0.027 16805 622. 16519 111 30
13 9 569 CASS IL 0.024 13437 560. 13384 16 8
14 10 570 CHAMPAIGN IL 0.058 173025 2983. 146506 16559 331
15 # 427 more rows
16 # 19 more variables: popasian <int>, popother <int>, percwhite <dbl>,
17 # percblack <dbl>, percamerindian <dbl>, percasian <dbl>, percother <dbl>,
18 # popadults <int>, perchs <dbl>, percollege <dbl>, percprof <dbl>,
19 # poppovertyknown <int>, percpovertyknown <dbl>, percbelowpoverty <dbl>,
20 # percchildbelowpovert <dbl>, percadultpoverty <dbl>, percelderlypoverty <dbl>,
21 # inmetro <int>, category <chr>
22 # Use `print(n = ...)` to see more rows
```

column types

abridged output

The Transform-Visualize-Model Cycle



Credit: Hadley Wickham

dplyr: A Package for Data Transformation



The dplyr data-transformation verbs we use:

1. Take a dataset as their first argument
2. Manipulate the dataset in some way
3. Returns the manipulated dataset

The Pipe

- Nested calls can be hard to read! (have to read inside out):

```
1 f(g(h(r(x))))
```

- The pipe `|>` allows us to move output between functions (i.e., `|>` = “and then”):

```
1 x |>  
2   r() |>  
3   h() |>  
4   g() |>  
5   f()
```

- The piped output goes to the first argument of the subsequent function by default.

Local News Data

- How does station ownership affect local news coverage?
- Martin and McCrain (2019) use data on local news at TV stations before and after a large acquisition by a conglomerate.

Variable	Description
callsign	Callsign of the station (i.e., unique station identifier)
affiliation	Network affiliation of the station (e.g., ABC, FOX News)
date	Airdate of news
weekday	Day of the week of airdate
ideology	Measure of news slant (bigger is more conservative)
national_politics	Avg proportion of segments on national politics
local_politics	Avg proportion of segments on local politics
sinclair2017	Station acquired by Sinclair group in Sept 2017
post	Date is before/after acquisition (0/1)

```

1 > # install the course data package (only once):
2 > # install.packages("https://j1yoo.github.io/assets/courses/pba/pbadata_0.1.0.tar.gz",
3 > #                               repos = NULL, type = "source")
4 > library(tidyverse)
5 > library(pbadata)
6 > news
7
8 # A tibble: 2,560 × 10
9   callsign affiliation date      weekday ideology national_politics local_politics
10  <chr>      <chr>      <date>      <ord>      <dbl>          <dbl>          <dbl>
11 1 KECI      NBC          2017-06-07 Wed         0.0655          0.225          0.148
12 2 KPAX      CBS          2017-06-07 Wed         0.0853          0.283          0.123
13 3 KRBC      NBC          2017-06-07 Wed         0.0183          0.130          0.189
14 4 KTAB      CBS          2017-06-07 Wed         0.0850          0.0901         0.138
15 5 KTMF      ABC          2017-06-07 Wed         0.0842          0.152          0.129
16 6 KTXS      ABC          2017-06-07 Wed        -0.000488       0.0925         0.0791
17 7 KAEF      ABC          2017-06-08 Thu         0.0426          0.213          0.228
18 8 KBVU      FOX          2017-06-08 Thu        -0.0860          0.169          0.247
19 9 KECI      NBC          2017-06-08 Thu         0.0902          0.276          0.152
20 10 KPAX     CBS          2017-06-08 Thu         0.0668          0.305          0.124
21 # i 2,550 more rows
22 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
23
24 > # Alternatively, import the CSV file and set the column types yourself:
25 > # news <- as_tibble(read.csv(url("https://bit.ly/3Qst0Ry")))
26 > # news <- news |>
27 > #   mutate(date = as.Date(date),
28 > #         weekday = factor(weekday, levels = c("Mon", "Tue", "Wed", "Thu", "Fri"),
29 > #                           ordered = TRUE),
30 > #         month = factor(month, levels = month.abb, ordered = TRUE))

```

1/ Operating on Rows

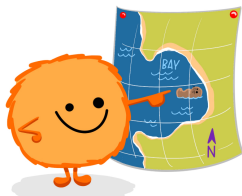
filter()

`filter()` selects rows that satisfy the argument you pass it:

`dplyr::filter()` KEEP ROWS THAT satisfy your CONDITIONS

keep rows from... this data... ONLY IF... type is "otter" AND site is "bay"

```
filter(df, type == "otter" & site == "bay")
```



type	food	site
otter	urchin	bay
shark	seal	channel
otter	abalone	bay
otter	crab	wharf



The table shows the results of the `filter()` operation. The first and third rows are highlighted in purple, indicating they satisfy the conditions. The second and fourth rows are crossed out with orange 'X' marks, indicating they do not satisfy the conditions. A purple cartoon character is standing on a green globe, pointing to the table. An orange cartoon character is pointing to a map of a bay.

Credit: Allison Horst

```

1 > news |>
2   filter(weekday == "Tue")
3
4 # A tibble: 509 × 10
5   callsign affiliation date      weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>      <dbl>      <dbl>
7 1 KAEF      ABC        2017-06-13 Tue        0.0242      0.180      0.234
8 2 KBVU      FOX        2017-06-13 Tue        0.00894     0.186      0.245
9 3 KBZK      CBS        2017-06-13 Tue        0.129      0.306      0.0763
10 4 KCVU      FOX        2017-06-13 Tue        0.114      0.124      0.178
11 5 KECI      NBC        2017-06-13 Tue        0.115      0.283      0.0926
12 6 KHSL      CBS        2017-06-13 Tue        0.0821     0.274      0.248
13 7 KNVN      NBC        2017-06-13 Tue        0.120      0.261      0.253
14 8 KPAX      CBS        2017-06-13 Tue        0.0984     0.208      0.171
15 9 KRCR      ABC        2017-06-13 Tue        0.0187     0.206      0.232
16 10 KTMF      ABC        2017-06-13 Tue        0.233      0.101      0.144
17 # i 499 more rows
18 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>

```

Multiple Conditions Means “and”!

```
1 > news |>
2   filter(weekday == "Tue",
3         affiliation == "FOX")
4
5 # A tibble: 70 × 10
6   callsign affiliation date       weekday ideology national_politics local_politics
7   <chr>      <chr>      <date>      <ord>      <dbl>      <dbl>      <dbl>
8 1 KBVU      FOX        2017-06-13 Tue        0.00894      0.186      0.245
9 2 KCVU      FOX        2017-06-13 Tue        0.114        0.124      0.178
10 3 WEMT      FOX        2017-06-13 Tue        0.235        0.149      0.155
11 4 WYDO      FOX        2017-06-13 Tue        0.0949       0.182      0.180
12 5 WEMT      FOX        2017-06-20 Tue        0.268        0.134      0.151
13 6 WYDO      FOX        2017-06-20 Tue        0.0590       0.155      0.0943
14 7 WEMT      FOX        2017-06-27 Tue        0.0457       0.164      0.171
15 8 WYDO      FOX        2017-06-27 Tue       -0.0588      0.147      0.123
16 9 WEMT      FOX        2017-07-04 Tue        0.0801       0.0986     0.180
17 10 WYDO      FOX        2017-07-04 Tue        0.0555       0.0910     0.169
18 # i 60 more rows
19 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

Common Mistake!

```
1 > news |>  
2   filter(weekday = "Tue")
```

Error in `filter()`:

! We detected a named input.

i This usually means that you've used `=` instead of `==`.

i Did you mean `weekday == "Tue"`?

Run `rlang::last\_trace()` to see where the error occurred.

Recall: Logicals

- Comparing two values/vectors:
 - $>/>=$: greater than/greater than or equal to
 - $</<=$: less than/less than or equal to
 - $==/!=$: equal to/not equal to
- Combining multiple logical statements:
 - $\&$: and (conjunction)
 - $|$: or (disjunction)

Filtering with Multiple Conditions: cond_a and cond_b

```
1 > news |>
2   filter(weekday == "Tue" & ideology < 0)
3
4 # A tibble: 66 × 10
5   callsign affiliation date       weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>      <dbl>      <dbl>
7 1 KAEF      ABC          2017-06-27 Tue      -0.0117      0.162      0.207
8 2 KECI      NBC          2017-06-27 Tue      -0.00362     0.177      0.0932
9 3 KHSL      CBS          2017-06-27 Tue      -0.0735     0.170      0.259
10 4 KNVN      NBC          2017-06-27 Tue      -0.0175     0.180      0.262
11 5 KPAX      CBS          2017-06-27 Tue      -0.134      0.219      0.120
12 6 KTXS      ABC          2017-06-27 Tue      -0.0307     0.129      0.100
13 7 WCTI      ABC          2017-06-27 Tue      -0.0308     0.187      0.119
14 8 WITN      NBC          2017-06-27 Tue      -0.0233     0.155      0.108
15 9 WJHL      CBS          2017-06-27 Tue      -0.00388    0.166      0.210
16 10 WNCT     CBS          2017-06-27 Tue      -0.130      0.181      0.125
17 # i 56 more rows
18 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

Filtering with Multiple Conditions: cond_1 or cond_2

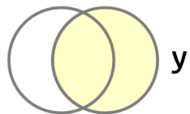
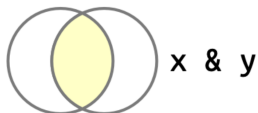
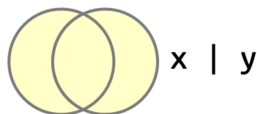
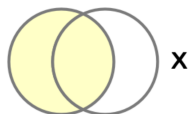
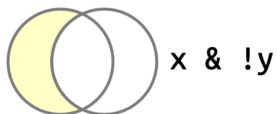
```
1 > news |>
2   filter(affiliation == "FOX" | affiliation == "ABC")
3
4 # A tibble: 1,033 × 10
5   callsign affiliation date       weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>          <dbl>          <dbl>
7 1 KTMF      ABC        2017-06-07 Wed         0.0842         0.152         0.129
8 2 KTXS      ABC        2017-06-07 Wed        -0.000488      0.0925      0.0791
9 3 KAEF      ABC        2017-06-08 Thu         0.0426         0.213         0.228
10 4 KBVU     FOX        2017-06-08 Thu        -0.0860         0.169         0.247
11 5 KTMF      ABC        2017-06-08 Thu         0.0433         0.179         0.139
12 6 KTXS      ABC        2017-06-08 Thu         0.0627         0.158         0.115
13 7 WCTI      ABC        2017-06-08 Thu         0.139          0.225         0.0759
14 8 KAEF      ABC        2017-06-09 Fri         0.0870         0.153         0.269
15 9 KTXS      ABC        2017-06-09 Fri         0.0879         0.0790         0.131
16 10 WCTI     ABC        2017-06-09 Fri         0.0667         0.182         0.111
17 # i 1,023 more rows
18 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

Combining %in%

- When you want to check if a value is equal to any one of several options, you can simplify it using %in%.
 - `a %in% B`, where $B = \{1,2,3\}$, is `a == 1 | a == 2 | a == 3`
 - Equivalent to 'combining' | and ==

```
1 > news |>
2   filter(weekday %in% c("Mon", "Fri"))
3
4 # A tibble: 1,021 × 10
5   callsign affiliation date      weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>          <dbl>          <dbl>
7 1 KAEF      ABC        2017-06-09 Fri        0.0870          0.153          0.269
8 2 KECI      NBC        2017-06-09 Fri        0.115          0.216          0.108
9 3 KPAX      CBS        2017-06-09 Fri        0.0882          0.315          0.128
10 4 KRBC      NBC        2017-06-09 Fri        0.0929          0.152          0.147
11 5 KTAB      CBS        2017-06-09 Fri        0.0588          0.0711         0.176
12 6 KTXS      ABC        2017-06-09 Fri        0.0879          0.0790         0.131
13 7 WCTI      ABC        2017-06-09 Fri        0.0667          0.182          0.111
14 8 WITN      NBC        2017-06-09 Fri       -0.0683          0.146          0.185
15 9 WNCT      CBS        2017-06-09 Fri       -0.0181          0.126          0.198
16 10 WYDO     FOX        2017-06-09 Fri       -0.0330          0.0802          0.194
17 # i 1,011 more rows
18 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```


Other Complicated Logicals



arrange()

`arrange()` will reorder the rows based on the values of the columns. With multiple arguments, sort by first argument, then second, then third...

Arrange by callsign then date

```
1 > news |>
2   arrange(callsign, date)
3
4 # A tibble: 2,560 × 10
5   callsign affiliation date      weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>          <dbl>          <dbl>
7 1 KAEF      ABC        2017-06-08 Thu         0.0426          0.213          0.228
8 2 KAEF      ABC        2017-06-09 Fri         0.0870          0.153          0.269
9 3 KAEF      ABC        2017-06-12 Mon         0.0135          0.149          0.188
10 4 KAEF      ABC        2017-06-13 Tue         0.0242          0.180          0.234
11 5 KAEF      ABC        2017-06-14 Wed         0.123           0.182          0.0968
12 6 KAEF      ABC        2017-06-15 Thu         0.0778          0.114          0.203
13 7 KAEF      ABC        2017-06-19 Mon         0.778           0.0823         0.179
14 8 KAEF      ABC        2017-06-20 Tue         0.115           0.131          0.163
15 9 KAEF      ABC        2017-06-21 Wed        -0.315           0.130          0.192
16 10 KAEF     ABC        2017-06-22 Thu         0.0873          0.104          0.189
17 # i 2,550 more rows
18 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

1/ Which Station-Date were the Most Liberal?

```
1 > news |>
2   arrange(ideology)
3
4 # A tibble: 2,560 × 10
5   callsign affiliation date      weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>          <dbl>          <dbl>
7 1 KRBC      NBC        2017-10-19 Thu        -0.674          0.0731          0.161
8 2 WJHL      CBS        2017-12-08 Fri        -0.673          0.0364          0.206
9 3 KRBC      NBC        2017-10-18 Wed        -0.586          0.0470          0.135
10 4 KCVU      FOX        2017-06-22 Thu        -0.414          0.158           0.172
11 5 KRBC      NBC        2017-12-11 Mon        -0.365          0.0674          0.163
12 6 KAEF      ABC        2017-06-21 Wed        -0.315          0.130           0.192
13 7 KTMF      ABC        2017-12-01 Fri        -0.303          0.179           0.150
14 8 KWYB      ABC        2017-12-01 Fri        -0.303          0.160           0.161
15 9 KTVM      NBC        2017-09-01 Fri        -0.302          0.0507          0.106
16 10 KNVN     NBC        2017-12-08 Fri        -0.299          0.121           0.211
17 # i 2,550 more rows
18 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

- Political affiliation: **Liberal** ↔ Conservative
 - Liberal: low ideology score
 - Conservative: high ideology score

1/ Which Station-Date were the Most Liberal?

```
1 > news |>
2   arrange(ideology)
3
4 # A tibble: 2,560 × 10
5   callsign affiliation date      weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>      <dbl>      <dbl>
7 1 KRBC      NBC        2017-10-19 Thu        -0.674      0.0731     0.161
8 2 WJHL      CBS        2017-12-08 Fri        -0.673      0.0364     0.206
9 3 KRBC      NBC        2017-10-18 Wed        -0.586      0.0470     0.135
10 4 KCVU      FOX        2017-06-22 Thu        -0.414      0.158      0.172
11 5 KRBC      NBC        2017-12-11 Mon        -0.365      0.0674     0.163
12 6 KAEF      ABC        2017-06-21 Wed        -0.315      0.130      0.192
13 7 KTMF      ABC        2017-12-01 Fri        -0.303      0.179      0.150
14 8 KWYB      ABC        2017-12-01 Fri        -0.303      0.160      0.161
15 9 KTVM      NBC        2017-09-01 Fri        -0.302      0.0507     0.106
16 10 KNVN     NBC        2017-12-08 Fri        -0.299      0.121      0.211
17 # i 2,550 more rows
18 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

- Political affiliation: **Liberal** ↔ Conservative
 - Liberal: low ideology score
 - Conservative: high ideology score



2/ Which Station-Date were the Most Conservative?

```
1 > news |>
2   arrange(desc(ideology))
3
4 # A tibble: 2,560 × 10
5   callsign affiliation date      weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>          <dbl>          <dbl>
7 1 KAEF      ABC        2017-06-19 Mon        0.778          0.0823         0.179
8 2 WYDO      FOX        2017-07-19 Wed        0.580          0.126          0.121
9 3 KRCR      ABC        2017-10-03 Tue        0.566          0.123          0.192
10 4 KAEF      ABC        2017-10-18 Wed        0.496          0.0892         0.217
11 5 KBVU      FOX        2017-11-16 Thu        0.491          0.159          0.184
12 6 KTMF      ABC        2017-11-06 Mon        0.455          0.138          0.154
13 7 KAEF      ABC        2017-06-29 Thu        0.447          0.126          0.220
14 8 KPAX      CBS        2017-11-23 Thu        0.437          0.125          0.128
15 9 KTAB      CBS        2017-11-16 Thu        0.427          0.0631         0.104
16 10 KCVU      FOX        2017-07-06 Thu        0.406          0.154          0.148
17 # i 2,550 more rows
18 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

- Political affiliation: Liberal ↔ Conservative
 - Liberal: low ideology score
 - Conservative: high ideology score

2/ Which Station-Date were the Most Conservative?

```
1 > news |>
2   arrange(desc(ideology))
3
4 # A tibble: 2,560 × 10
5   callsign affiliation date      weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>          <dbl>          <dbl>
7 1 KAEF      ABC        2017-06-19 Mon         0.778          0.0823         0.179
8 2 WYDO      FOX        2017-07-19 Wed         0.580          0.126          0.121
9 3 KRCR      ABC        2017-10-03 Tue         0.566          0.123          0.192
10 4 KAEF      ABC        2017-10-18 Wed         0.496          0.0892         0.217
11 5 KBVU      FOX        2017-11-16 Thu         0.491          0.159          0.184
12 6 KTMF      ABC        2017-11-06 Mon         0.455          0.138          0.154
13 7 KAEF      ABC        2017-06-29 Thu         0.447          0.126          0.220
14 8 KPAX      CBS        2017-11-23 Thu         0.437          0.125          0.128
15 9 KTAB      CBS        2017-11-16 Thu         0.427          0.0631         0.104
16 10 KCVU      FOX        2017-07-06 Thu         0.406          0.154          0.148
17 # i 2,550 more rows
18 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

- Political affiliation: Liberal ↔ Conservative
 - Liberal: low ideology score
 - Conservative: high ideology score



slice()

- `slice()` can give you a specific set of rows:

```
1 > news |>
2   slice(1,3)
3
4 # A tibble: 2 × 10
5   callsign affiliation date       weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>          <dbl>          <dbl>
7 1 KECI      NBC          2017-06-07 Wed         0.0655          0.225          0.148
8 2 KRBC      NBC          2017-06-07 Wed         0.0183          0.130          0.189
9 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

- You can ask for a range of rows with `start:stop` syntax:

```
1 > news |>
2   slice(1:3)
3
4 # A tibble: 3 × 10
5   callsign affiliation date       weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>          <dbl>          <dbl>
7 1 KECI      NBC          2017-06-07 Wed         0.0655          0.225          0.148
8 2 KPAX      CBS          2017-06-07 Wed         0.0853          0.283          0.123
9 3 KRBC      NBC          2017-06-07 Wed         0.0183          0.130          0.189
10 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```


slice_min() & slice_max()

- `slice_min(var, n = k)` will return the bottom k observations on column `var`:
 - Likewise, `slice_max(var, n = k)` will return the top k obs.

```
1 > news |>
2   slice_min(ideology, n = 5)
3
4 # A tibble: 5 × 10
5   callsign affiliation date       weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>         <dbl>         <dbl>
7 1 KRBC      NBC        2017-10-19 Thu        -0.674         0.0731         0.161
8 2 WJHL      CBS        2017-12-08 Fri        -0.673         0.0364         0.206
9 3 KRBC      NBC        2017-10-18 Wed        -0.586         0.0470         0.135
10 4 KCVU      FOX        2017-06-22 Thu        -0.414         0.158          0.172
11 5 KRBC      NBC        2017-12-11 Mon        -0.365         0.0674         0.163
12 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

2/ Operating on Columns

select()

- `select()` selects columns via their names.
- Select based on names (left) or based on a range of variables (right):

```
1 > news |>
2   select(callsign, date, ideology)
3
4 # A tibble: 2,560 × 3
5   callsign date      ideology
6   <chr>   <date>    <dbl>
7 1 KECI   2017-06-07  0.0655
8 2 KPAX   2017-06-07  0.0853
9 3 KRBC   2017-06-07  0.0183
10 4 KTAB   2017-06-07  0.0850
11 5 KTMF   2017-06-07  0.0842
12 6 KTXS   2017-06-07 -0.000488
13 7 KAEF   2017-06-08  0.0426
14 8 KBVU   2017-06-08 -0.0860
15 9 KECI   2017-06-08  0.0902
16 10 KPAX   2017-06-08  0.0668
17 # i 2,550 more rows
```

```
1 > news |>
2   select(callsign:date)
3
4 # A tibble: 2,560 × 3
5   callsign affiliation date
6   <chr>    <chr>    <date>
7 1 KECI     NBC     2017-06-07
8 2 KPAX     CBS     2017-06-07
9 3 KRBC     NBC     2017-06-07
10 4 KTAB     CBS     2017-06-07
11 5 KTMF     ABC     2017-06-07
12 6 KTXS     ABC     2017-06-07
13 7 KAEF     ABC     2017-06-08
14 8 KBVU     FOX     2017-06-08
15 9 KECI     NBC     2017-06-08
16 10 KPAX     CBS     2017-06-08
17 # i 2,550 more rows
```

1/ More Use Cases of select()

- Selecting all that are not (i.e., !) in a given range:

```
1 > news |>
2   select(!callsign:date)
3
4 # A tibble: 2,560 × 7
5   weekday ideology national_politics local_politics sinclair2017 post month
6   <ord>      <dbl>          <dbl>          <dbl>          <int> <int> <ord>
7 1 Wed      0.0655          0.225          0.148           1     0 Jun
8 2 Wed      0.0853          0.283          0.123           0     0 Jun
9 3 Wed      0.0183          0.130          0.189           0     0 Jun
10 4 Wed      0.0850          0.0901         0.138           0     0 Jun
11 5 Wed      0.0842          0.152          0.129           0     0 Jun
12 6 Wed     -0.000488        0.0925          0.0791          1     0 Jun
13 7 Thu      0.0426          0.213          0.228           1     0 Jun
14 8 Thu     -0.0860          0.169          0.247           1     0 Jun
15 9 Thu      0.0902          0.276          0.152           1     0 Jun
16 10 Thu     0.0668          0.305          0.124           0     0 Jun
17 # i 2,550 more rows
```

2/ More Use Cases of select()

- Selecting all numeric columns:
 - `dplyr::where()` takes a condition and returns a list of variables that satisfy it.

```
1 > news |>
2   select(where(is.numeric))
3
4 # A tibble: 2,560 × 5
5   ideology national_politics local_politics sinclair2017 post
6   <dbl>         <dbl>         <dbl>         <int> <int>
7 1  0.0655         0.225         0.148           1     0
8 2  0.0853         0.283         0.123           0     0
9 3  0.0183         0.130         0.189           0     0
10 4  0.0850         0.0901        0.138           0     0
11 5  0.0842         0.152         0.129           0     0
12 6 -0.000488       0.0925        0.0791          1     0
13 7  0.0426         0.213         0.228           1     0
14 8 -0.0860         0.169         0.247           1     0
15 9  0.0902         0.276         0.152           1     0
16 10 0.0668         0.305         0.124           0     0
17 # i 2,550 more rows
```

3/ More Use Cases of select()

- Combining multiple selections:
 - dplyr::ends_with() selects columns whose names end with a specified string.
 - Similarly, dplyr::starts_with() (or dplyr::contains()) points to columns with names that begin with (or contains) a specified string,

```
1 > news |>
2   select(callsign:weekday, ends_with("politics"))
3
4 # A tibble: 2,560 × 6
5   callsign affiliation date      weekday
6   <chr>      <chr>      <date>    <ord>
7 1 KECI      NBC        2017-06-07 Wed
8 2 KPAX      CBS        2017-06-07 Wed
9 3 KRBC      NBC        2017-06-07 Wed
10 4 KTAB      CBS        2017-06-07 Wed
11 5 KTMF      ABC        2017-06-07 Wed
12 6 KTXS      ABC        2017-06-07 Wed
13 7 KAEF      ABC        2017-06-08 Thu
14 8 KBVU      FOX        2017-06-08 Thu
15 9 KECI      NBC        2017-06-08 Thu
16 10 KPAX     CBS        2017-06-08 Thu
17 # i 2,550 more rows
18 # i 2 more variables: national_politics <dbl>,
19 #   local_politics <dbl>
```

rename()

- `rename(new_name = old_name)` renames the `old_name` variable to `new_name`.

```
1 > news |>
2   rename(call_sign = callsign)
3
4 # A tibble: 2,560 × 10
5   call_sign affiliation date      weekday ideology national_politics local_politics
6   <chr>      <chr>      <date>      <ord>      <dbl>      <dbl>      <dbl>
7 1 KECI      NBC        2017-06-07 Wed        0.0655      0.225      0.148
8 2 KPAX      CBS        2017-06-07 Wed        0.0853      0.283      0.123
9 3 KRBC      NBC        2017-06-07 Wed        0.0183      0.130      0.189
10 4 KTAB      CBS        2017-06-07 Wed        0.0850      0.0901     0.138
11 5 KTMF      ABC        2017-06-07 Wed        0.0842      0.152      0.129
12 6 KTXS      ABC        2017-06-07 Wed       -0.000488    0.0925     0.0791
13 7 KAEF      ABC        2017-06-08 Thu        0.0426      0.213      0.228
14 8 KBVU      FOX        2017-06-08 Thu       -0.0860      0.169      0.247
15 9 KECI      NBC        2017-06-08 Thu        0.0902      0.276      0.152
16 10 KPAX     CBS        2017-06-08 Thu        0.0668      0.305      0.124
17 # i 2,550 more rows
18 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

mutate()

- `mutate(new_var = fun(old_vars))` adds new columns that are functions of existing columns.

```
1 > news |>
2   mutate(
3     national_local_diff = national_politics - local_politics,
4     national_politics_perc = national_politics * 100
5   ) |>
6   select(callsign, date, national_politics, local_politics,
7          national_local_diff, national_politics_perc)
8
9 # A tibble: 2,560 × 6
10   callsign date      national_politics local_politics national_local_diff
11   <chr>    <date>          <dbl>          <dbl>          <dbl>
12 1 KECI    2017-06-07        0.225          0.148          0.0761
13 2 KPAX    2017-06-07        0.283          0.123          0.160
14 3 KRBC    2017-06-07        0.130          0.189         -0.0589
15 4 KTAB    2017-06-07        0.0901         0.138         -0.0476
16 5 KTMF    2017-06-07        0.152          0.129          0.0229
17 6 KTXS    2017-06-07        0.0925         0.0791         0.0134
18 7 KAEF    2017-06-08        0.213          0.228         -0.0151
19 8 KBVU    2017-06-08        0.169          0.247         -0.0781
20 9 KECI    2017-06-08        0.276          0.152          0.124
21 10 KPAX    2017-06-08        0.305          0.124          0.180
22 # i 2,550 more rows
23 # i 1 more variable: national_politics_perc <dbl>
```


if_else()

- `if_else(condition, true, false)` creates a vector based on a logical condition.
 - Returns the true value when the condition is TRUE, the false value when FALSE, and a missing value when NA.

```
1 > news |>
2   mutate(Ownership = if_else(sinclair2017 == 1,
3                             "Acquired by Sinclair",
4                             "Not Acquired")) |>
5   select(callsign, affiliation, date, Ownership)
6
7 # A tibble: 2,560 × 4
8   callsign affiliation date      Ownership
9   <chr>      <chr>      <date>      <chr>
10 1 KECI      NBC        2017-06-07 Acquired by Sinclair
11 2 KPAX      CBS        2017-06-07 Not Acquired
12 3 KRBC      NBC        2017-06-07 Not Acquired
13 4 KTAB      CBS        2017-06-07 Not Acquired
14 5 KTMF      ABC        2017-06-07 Not Acquired
15 6 KTXS      ABC        2017-06-07 Acquired by Sinclair
16 7 KAEF      ABC        2017-06-08 Acquired by Sinclair
17 8 KBVU      FOX        2017-06-08 Acquired by Sinclair
18 9 KECI      NBC        2017-06-08 Acquired by Sinclair
19 10 KPAX     CBS        2017-06-08 Not Acquired
20 # i 2,550 more rows
```

3/ Operating on Groups

group_by()

- `group_by(var)` divides the data into groups based on the `var` variable:
 - Doesn't change the data yet, but subsequent operations will be done by groups specified in `var`.

```
1 > news |>
2   group_by(month)
3
4 # A tibble: 2,560 × 10
5 # Groups:   month [7]
6   callsign affiliation date      weekday ideology national_politics local_politics
7   <chr>      <chr>      <date>      <ord>      <dbl>      <dbl>      <dbl>
8 1 KECI      NBC        2017-06-07 Wed        0.0655      0.225      0.148
9 2 KPAX      CBS        2017-06-07 Wed        0.0853      0.283      0.123
10 3 KRBC      NBC        2017-06-07 Wed        0.0183      0.130      0.189
11 4 KTAB      CBS        2017-06-07 Wed        0.0850      0.0901     0.138
12 5 KTMF      ABC        2017-06-07 Wed        0.0842      0.152      0.129
13 6 KTXS      ABC        2017-06-07 Wed       -0.000488   0.0925     0.0791
14 7 KAEF      ABC        2017-06-08 Thu        0.0426      0.213      0.228
15 8 KBVU      FOX        2017-06-08 Thu       -0.0860     0.169      0.247
16 9 KECI      NBC        2017-06-08 Thu        0.0902      0.276      0.152
17 10 KPAX     CBS        2017-06-08 Thu        0.0668      0.305      0.124
18 # i 2,550 more rows
19 # i 3 more variables: sinclair2017 <int>, post <int>, month <ord>
```

summarize()

- `summarize(sum_var = fun(curr_var))` calculates summaries of variables:

```
1 > news |>
2   group_by(month) |>
3   summarize(
4     political_leaning_mean = mean(ideology, na.rm = TRUE)
5   )
6
7 # A tibble: 7 × 2
8   month political_leaning_mean
9   <ord>          <dbl>
10 1 Jun           0.0786
11 2 Jul           0.103
12 3 Aug           0.105
13 4 Sep           0.0751
14 5 Oct           0.0862
15 6 Nov           0.0972
16 7 Dec           0.0774
```

Summaries by Ownership and Pre/Post

```
1 > news |>
2   group_by(sinclair2017, post) |>
3   summarize(
4     slant_mean = mean(ideology, na.rm = TRUE),
5     national_mean = mean(national_politics, na.rm = TRUE)
6   )
7
8 # A tibble: 4 × 4
9 # Groups:   sinclair2017 [2]
10  sinclair2017  post slant_mean national_mean
11      <int> <int>      <dbl>      <dbl>
12 1         0     0      0.100      0.134
13 2         0     1      0.0768     0.126
14 3         1     0      0.0936     0.137
15 4         1     1      0.0938     0.155
```

Summaries Across Types of Variables

- `across()` will apply a summary function across many variables:

```
1 > news |>
2   group_by(sinclair2017, post) |>
3   summarize(
4     across(where(is.numeric), ~ mean(.x, na.rm = TRUE))
5   )
6
7 # A tibble: 4 × 5
8 # Groups:   sinclair2017 [2]
9   sinclair2017 post ideology national_politics local_politics
10    <int> <int>    <dbl>          <dbl>          <dbl>
11 1         0  0    0.100            0.134            0.168
12 2         0  1    0.0768           0.126            0.167
13 3         1  0    0.0936           0.137            0.157
14 4         1  1    0.0938           0.155            0.139
```

kable() to produce nice tables

```
1 > news |>  
2   group_by(month) |>  
3   summarize(  
4     political_leaning_mean = mean(ideology, na.rm = TRUE) ) |>  
5   knitr::kable(col.names = c("Month", "Avg. Ideology")) # we can also specify column names
```

Month	Avg. Ideology
Jun	0.0785518
Jul	0.1032917
Aug	0.1049908
Sep	0.0751067
Oct	0.0861639
Nov	0.0971796
Dec	0.0773873

Producing a Table of Counts for a Categorical Var.

- First begin by `group_by()`, then `summarize(n = n())` to count the observations, or...

```
1 > news |>
2   group_by(affiliation) |>
3   summarize(n = n())
4
5 # A tibble: 4 × 2
6   affiliation      n
7   <chr>         <int>
8 1 ABC           687
9 2 CBS           758
10 3 FOX           346
11 4 NBC           769
```

- Just use `count()`!

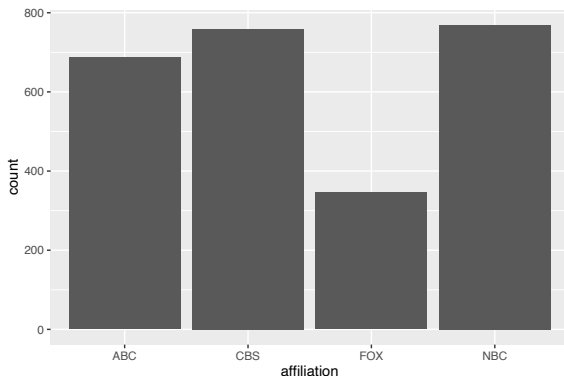
```
1 > news |>
2   count(affiliation)
3
4 # A tibble: 4 × 2
5   affiliation      n
6   <chr>         <int>
7 1 ABC           687
8 2 CBS           758
9 3 FOX           346
10 4 NBC           769
```


4/ Creating Barplots

Combining Our Skills

- Let's combine our tools to produce a bar plot with `geom_bar()`.
- Recall: By default, bar plots take a single variable and show the number of observations in each category.

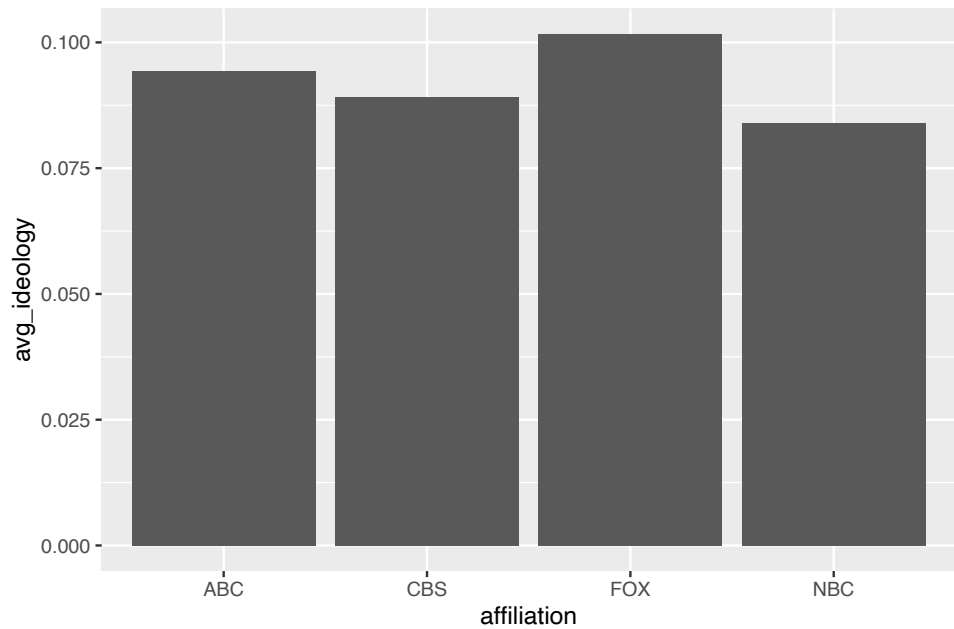
```
1 > ggplot(news, aes(x = affiliation)) +  
2   geom_bar()
```



Barplots of Non-Counts

- Barplots can represent a lot beyond counts, including variables in our dataset or group summaries.
 - When the height of the bar is another variable in our data and not just a count, we set the x and y aesthetics and use `geom_col()` instead of `geom_bar()`.
- Let's create a group summary then plot it:

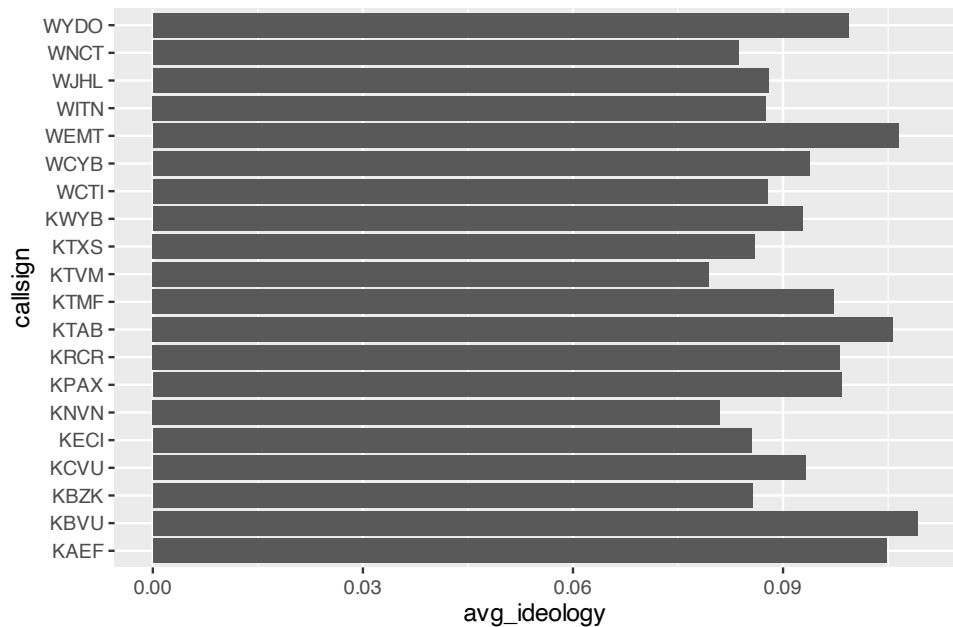
```
1 > aff_ideology_means <- news |>
2   group_by(affiliation) |>
3   summarize(avg_ideology = mean(ideology, na.rm = TRUE))
4 > aff_ideology_means
5
6 # A tibble: 4 × 2
7   affiliation avg_ideology
8   <chr>         <dbl>
9 1 ABC           0.0943
10 2 CBS           0.0891
11 3 FOX           0.102
12 4 NBC           0.0841
13
14
15 > ggplot(aff_ideology_means, aes(x = affiliation, y = avg_ideology)) +
16   geom_col()
```



Slightly More Complicated Example

- Let's create a barplot that shows the top 20 stations in terms of their political leaning (i.e., ideology).
- First, let's get the data then plot:

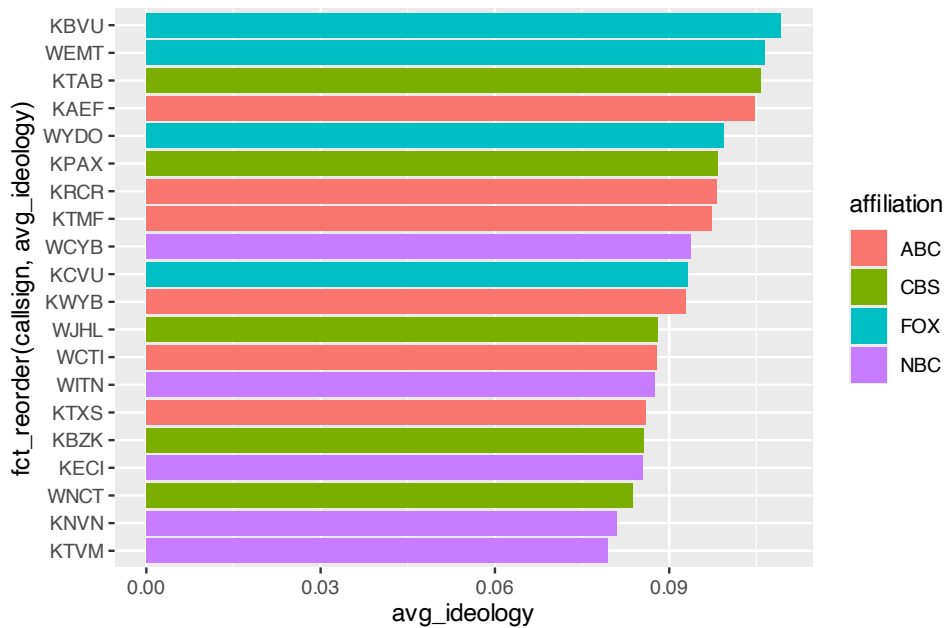
```
1 > station_ideology <- news |>
2   group_by(callsign, affiliation) |>
3   summarize(avg_ideology = mean(ideology, na.rm = TRUE), .groups = "drop") |>
4   slice_max(avg_ideology, n = 20)
5
6 > ggplot(station_ideology, aes(x = avg_ideology, y = callsign)) +
7   geom_col()
```



How Do We Reorder the Stations?

- We'd like to order the stations by ideology.
- `fct_reorder`(group, order_var), loaded with tidyverse, reorders factor levels by the ordering variable (low to high).
 - Here we use it inside the mapping (i.e., `aes()`).

```
1 > ggplot(data = station_ideology,  
2       mapping = aes(x = avg_ideology,  
3                     y = fct_reorder(callsign, avg_ideology))) +  
4       geom_col(aes(fill = affiliation))
```



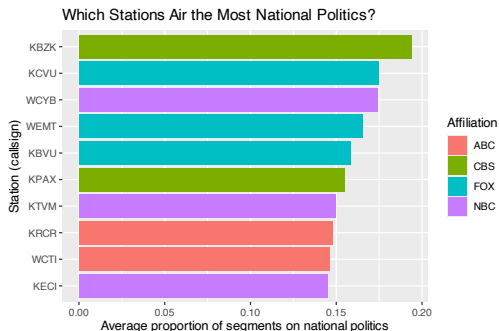
5/ In-Class Exercise E2

In-Class Exercises: Background

- We want to find out the answer to the following question:
 - Q: which local TV stations devote the most coverage to national politics?
- The news data from today's class can answer this, so keep working in the same R session.
 - Some relevant variables:
 - `callsign`: unique station identifier
 - `affiliation`: network affiliation of the station
 - `national_politics`: average proportion of segments on national politics on that airdate

In-Class Exercises: Task

- Recreate the following plot:



- The plot shows the 10 stations with the highest average proportion of segments on national politics, ordered from highest to lowest and colored by network affiliation.
 - To get started, we'll give you a few clues on the following slide.

In-Class Exercises: Instructions

- For each station, compute the average of `national_politics` and the number of airdates behind it (`n = n()`). Keep affiliation in the table.
- Keep the 10 stations with the highest average, and draw the bars in that order.
- Make sure that the labels are correctly specified.
- **Verify before you trust the output:**
 - How many rows does your summary table have? It must be 10. If you get 22, find out why and fix it.
 - Recompute the top station's average a different way, with `filter()` and then `summarize()`. Does it match your bar?

Q: What does the plot reveal? Which bar would you trust the least, and why?
(Hint: look at `n`.)

What Comes Next?

- Today: rows, columns, and groups with `dplyr`, then barplots drawn from the summaries.
- No class next Monday, Sep 28 (Teachers' Day). Use the week for Assignment 1; TAs hold office hours as usual.
- Next class (Oct 5): **causality**. What is a causal effect, randomized experiments, and observational studies. Read IMS §§2.2–2.3 beforehand.
- **Assignment 1 is released today**. You have two weeks: it is due before class starts on Monday, Oct 5 (undergraduate).
- **E2 (today's exercise): submit on eeclass by 23:59 today**.

Enjoy your Weekends! :)

Contact Information:

jaewon.yoo@iss.nthu.edu.tw

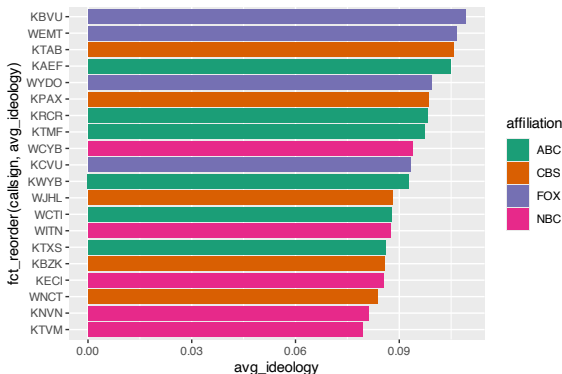
<https://j1yoo.github.io/>



Setting the Color Palette

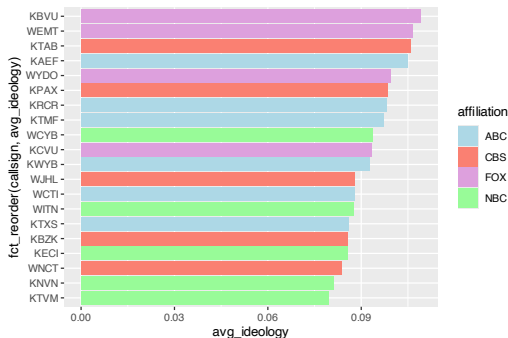
- We can use color palettes from a 'project'/built-in package called ColorBrewer:

```
1 > ggplot(station_ideology,  
2       mapping = aes(x = avg_ideology,  
3                     y = fct_reorder(callsign, avg_ideology))) +  
4   geom_col(mapping = aes(fill = affiliation)) +  
5   scale_fill_brewer(palette = "Dark2")
```



Manually Setting the Color Palette

```
1 > ggplot(station_ideology,  
2   mapping = aes(x = avg_ideology,  
3     y = fct_reorder(callsign, avg_ideology))) +  
4   geom_col(mapping = aes(fill = affiliation)) +  
5   scale_fill_manual(values = c(ABC = "lightblue",  
6     CBS = "salmon",  
7     FOX = "plum",  
8     NBC = "palegreen"))
```



Fun with Colors

```
1 > library(wesanderson) # install the package first!
2 > ggplot(station_ideology,
3       mapping = aes(x = avg_ideology,
4                     y = fct_reorder(callsign, avg_ideology))) +
5   geom_col(mapping = aes(fill = affiliation)) +
6   scale_fill_manual(values = wes_palette("Moonrise3"))
```

